

Instituto

Universidad Veracruzana - Región Xalapa

Aplicación web MVC con Node JS

Desarrollo de Sistemas Web -
Erika Meneses Rico

Abraham Nuñez Sanchez

Mayo 2026

Captura 1: Base de datos MariaDB

Se observa la base de datos del proyecto con el gestor de base de datos MariaDB ejecutando consultas SQL sobre la base de datos llamada “clases”.

Esto evidencia que la base de datos contiene información relacional entre alumnos y materias mediante el campo id_materia.

```
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| clases   |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
5 rows in set (0.001 sec)

MariaDB [(none)]> use clases;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [clases]> show tables;
+-----+
| Tables_in_clases |
+-----+
| alumno           |
| materia          |
+-----+
2 rows in set (0.001 sec)
```

Se ejecuta el comando:

```
- select * from alumno;
```

Como resultado se muestra una tabla con bordes y columnas:

- id
- nombre
- promedio
- id_materia

La tabla contiene 11 registros con nombres completos de alumnos, promedios numéricos e identificadores de materia. Debajo se ejecuta el comando:

- select * from materia;

Se despliega otra tabla con las columnas:

- id
- nombre

```
MariaDB [clases]> select * from alumno;
```

id	nombre	promedio	id_materia
1	Sofia Martinez Lopez	9.2	5
2	Miguel Angel Torres Ruiz	8.5	6
3	Carolina Herrera Gutierrez	9.8	4
4	Laura Gonzales Sanchez	8	7
5	Ricardo Fernandez Moreno	7.8	6
6	Ana Maria Sanchez Martin	9.5	1
7	Juan Carlos Navarro Jimenez	6.9	1
8	Beatriz Molina Castro	8.3	2
9	Fernando Gomez Velasco	7.6	2
10	Jose Luis Rodriguez Perez	7.4	3
11	Claudia Jimenez Rivas	9.1	3

```
11 rows in set (0.001 sec)
```



```
MariaDB [clases]> select * from materia;
```

id	nombre
1	Arquitecturas de Red
2	Sistemas Operativos
3	Estructuras de Datos
4	Bases de Datos
5	Desarrollo de Sistemas Web
6	Enrutamiento Avanzado
7	Computo en la Nube
8	Programacion Seguro
9	Programacion en Red

```
9 rows in set (0.001 sec)
```



```
MariaDB [clases]>
```

Captura 2: Interfaz web del proyecto

Se muestra la aplicación web ejecutándose localmente en: <http://localhost:3000>

En la parte superior centrada aparece el título:

- Alumnos con materias

Debajo se encuentra el subtítulo:

- Lista de alumnos

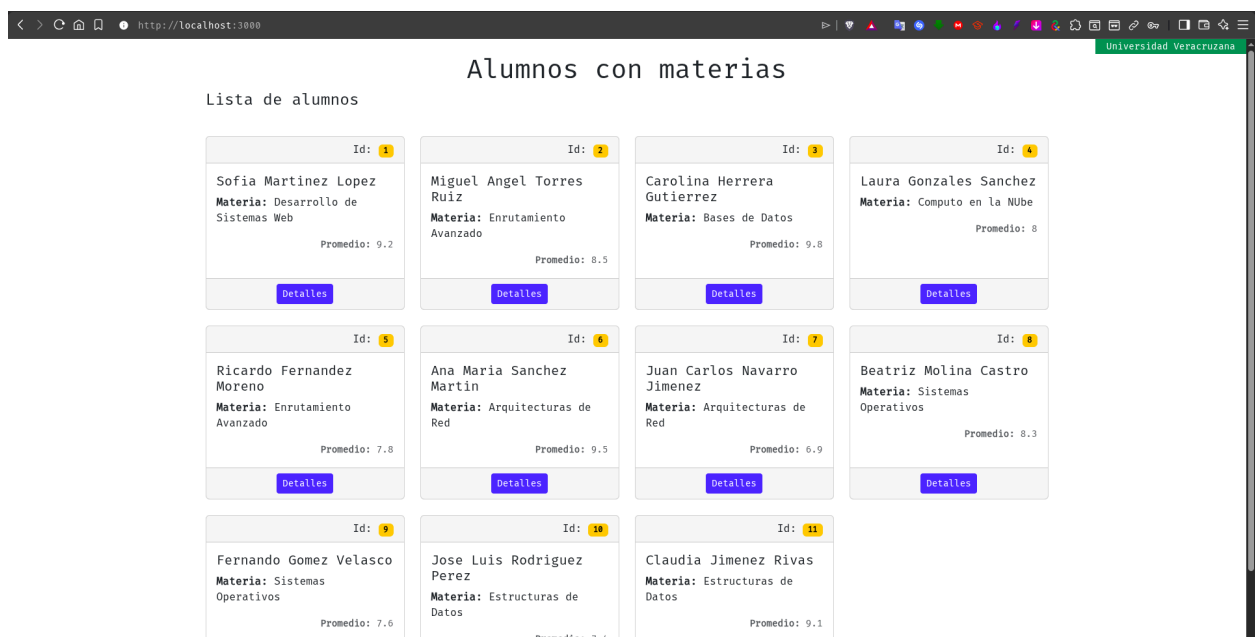
En la esquina superior derecha se observa una barra verde con texto blanco que dice:

- Universidad Veracruzana

En el contenido principal aparecen varias tarjetas blancas organizadas en filas y columnas. Cada tarjeta contiene:

- texto con el identificador del alumno: Id
- nombre del alumno
- texto Materia: seguido del nombre de una materia
- texto Promedio: seguido de un número decimal

En la parte inferior de cada tarjeta aparece un botón azul con el texto: “Detalles”



Captura 3: Archivo principal del servidor (index.js)

Se observa el editor Visual Studio Code abierto sobre el archivo [index.js](#).

El archivo contiene código JavaScript. En las primeras líneas se observan importaciones como:

- `const express = require('express')`
- `const dotenv = require('dotenv')`

Más abajo aparecen líneas relacionadas con configuración de vistas, rutas, middleware y servidor.

El código muestra la configuración principal de una aplicación en Node.js con Express. La captura evidencia la estructura base del servidor backend y configuración del framework.

```
JS index.js > ...
1  const express = require('express')
2  const dotenv = require('dotenv')
3  const app = express()
4
5  //primero carga la configuracion del archivo .env
6  //para que este disponible en las demas llamadas
7  dotenv.config();
8  //establece el motor de vistas a ejs
9  app.set('view engine', 'ejs')
10
11 //ruta de la URL raiz
12 const homeRouter = require('./routes/home')
13 app.use("/", homeRouter)
14 app.use('/home', homeRouter)
15
16 //middleware para el manejo de errores (debe de ser el ultimo middleware a utilizar)
17 const errorHandler = require('./middlewares/errorhandler')
18 app.use(errorHandler)
19
20 //inicia el servidor web en el puerto SERVER_PORT
21 app.listen(process.env.SERVER_PORT, () => {
22 |   console.log(`Aplicacion de ejemplo escuchando en el puerto ${process.env.SERVER_PORT}`)
23 | })
```

Captura 4: Estructura del proyecto MVC

Se muestra el explorador de archivos del proyecto llamado DSW-A12 dentro de Visual Studio Code.

En la parte superior aparece el nombre del proyecto:

- DSW-A12

Se observan carpetas desplegadas:

- controllers
- data
- middlewares
- models
- routes
- views/home

Dentro de ellas se muestran archivos:

- home.js
- datacontext.js
- errorhandler.js
- alumno.js
- _partialcard.ejs
- index.ejs

También se observan otros archivos:

- .env
- bd.sql
- index.js
- package-lock.json
- package.json

Esto demuestra una organización modular del proyecto, separando lógica de negocio, rutas, vistas, configuración y acceso a datos.

DSW-A12	
controllers	
JS home.js	U
data	
JS datacontext.js	U
middlewares	
JS errorhandler.js	U
models	
JS alumno.js	U
node_modules	
routes	
JS home.js	U
views / <u>home</u>	
<> _partialcard.ejs	
<> index.ejs	U
⚙ .env	U
📄 bd.sql	U
JS index.js	U
{ } package-lock.json	U
{ } package.json	U